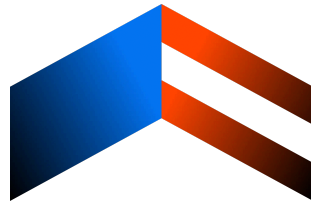


SECURITY REVIEW

FOR MANIFEST FINANCE



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	C-01 First Deposit Can Result in Zero Shares Due to Direct Token Transfer	09
	H-01 FULL-Restricted Users Can Still Deposit	10
	M-01 _currentDefaultAdmin Not set in SingleAdminAccessControl	11
	L-01 Pause bypass for approvals via permit	12
	L-02 Burner role blocked by Auth Manager checks	13
	I-01 chainalysisOracleEnabled() function logic inverted	14
08	DISCLAIMER	15
09	ABOUT KANN AUDITS	16

AUDIT SUMMARY

Manifest Finance engaged Kann Audits to perform a security review of the Manifest Finance smart contracts. A team of 2 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

6
ISSUES

CRITICAL	01
HIGH	01
MEDIUM	01
LOW	02
INFORMATIONAL	01

ENGAGEMENT DETAILS

PREPARED FOR Manifest Finance

PREPARED BY Kann Audits

DATE REPORTED 26 August 2025

METHODS Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Private
AUDITED COMMIT	Private

#	FILE
01	/src/authmanager/AuthManager.sol
02	/src/authmanager/AuthManagerStorage.sol
03	/src/chainalysis/MockChainalysisOracle.sol
04	/src/chainalysis/SanctionsList.sol
05	/src/chainalysis/SanctionsListStorage.sol
06	/src/oracle/USHPriceOracle.sol
07	/src/oracle/USHPriceOracleStorage.sol
08	/src/sush/AdminControl.sol
09	/src/sush/Silo.sol
10	/src/sush/StakedUSH.sol
11	/src/sush/StakedUSHBase.sol
12	/src/ush/USAToken.sol
13	/src/ush/USHToken.sol
14	/src/ush/USHTokenStorage.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

LEAD SECURITY RESEARCHER

RA

Radev_eth

LEAD SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
01	01	01	02	01
TOTAL 06 FIXED 06 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
C-01	First Deposit Can Result in Zero Shares Due to Direct Token Transfer	● CRITICAL	● FIXED
H-01	FULL-Restricted Users Can Still Deposit	● HIGH	● FIXED
M-01	_currentDefaultAdmin Not set in SingleAdminAccessControl	● MEDIUM	● FIXED
L-01	Pause bypass for approvals via permit	● LOW	● FIXED
L-02	Burner role blocked by Auth Manager checks	● LOW	● FIXED
I-01	chainalysisOracleEnabled() function logic inverted	● INFORMATIONAL	● FIXED

DETAILED FINDINGS

C-01

First Deposit Can Result in Zero Shares Due to Direct Token Transfer

SEVERITY

● CRITICAL

STATUS

● FIXED

■ DESCRIPTION

The StakedUSH vault is vulnerable to zero share deposits when USH tokens are sent directly to the contract. Because the vault calculates shares based on totalSupply and totalAssets, a direct token transfer increases totalAssets without increasing totalSupply. As a result, legitimate depositor will receive 0 shares for a positive deposit, breaking fair share distribution and potentially locking user funds.

The sUSH vault calculates shares using the formula:

$$\text{shares} = (\text{assets} * (\text{totalSupply} + \text{decimalsOffset})) / (\text{totalAssets} + 1)$$

If a user directly transfers USH to the vault contract (bypassing deposit()), the vault's totalAssets increases while totalSupply remains 0.

When the next user calls deposit(), the formula becomes:

$$(\text{assets} * (0 + 0)) / (\text{totalAssets} + 1) = 0$$

Thus, the user receives 0 shares (REVERTS), even though they tried depositing a positive amount of assets.

■ REMEDIATION

● FIXED

H-01

FULL-Restricted Users Can Still Deposit

SEVERITY

● HIGH

STATUS

● FIXED

■ DESCRIPTION

The deposit flow does not fully enforce the FULL_RESTRICTED_STAKER_ROLE check. When a user with FULL restriction calls deposit/assets, receiver, the restriction logic fails to prevent the operation, allowing restricted users to bypass intended compliance rules.

Deposit flow: deposit(ERC4626 logic) → _deposit(override) where checks if both addresses have SOFT_RESTRICTED_STAKER_ROLE if true reverts then logic goes to _deposit(ERC4626)

```
function _deposit(address caller, address receiver, uint256 assets, uint256 shares)
    internal virtual {

        SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
        _mint(receiver, shares);

        emit Deposit(caller, receiver, assets, shares);
    }
```

Where _mint is called.

```
function _mint(address account, uint256 value) internal {
    if (account == address(0)) {
        revert ERC20InvalidReceiver(address(0));
    }
    _update(address(0), account, value);
}
```

See how _mint sets first parameter for _update to be address 0.

and since we have _update override it goes to

```
function _update(address from, address to, uint256 value) internal override {
    if (hasRole(FULL_RESTRICTED_STAKER_ROLE, from) && to != address(0)) {
        revert OperationNotAllowed();
    }
    if (hasRole(FULL_RESTRICTED_STAKER_ROLE, to)) {
        revert OperationNotAllowed();
    }
    super._update(from, to, value);
}
```

Where it fails to check if the msg.sender is restricted role

■ REMEDIATION

● FIXED

M-01

_currentDefaultAdmin Not set in SingleAdminAccessControl

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

In StakedUshBase.sol, which imports and inherits AdminControl, the constructor calls:

```
_grantRole(DEFAULT_ADMIN_ROLE, _owner);
```

This sets the default admin role to _owner. However, the contract does not initialize the _currentDefaultAdmin variable, leaving it as address(0).

Consequently, in a scenario where the current default admin attempts to transfer the admin role by calling transferAdmin, followed by _pendingDefaultAdmin executing acceptAdmin, the _grantRole override logic executes:

```
_revokeRole(DEFAULT_ADMIN_ROLE, _currentDefaultAdmin);
```

Since _currentDefaultAdmin is address(0), the call to _revokeRole effectively does nothing. The function then proceeds to grant the role to the new admin via:

```
super._grantRole(role, account);
```

The net effect is that both the original _owner and the new admin retain the DEFAULT_ADMIN_ROLE, resulting in two admins instead of a proper transfer.

This issue arises from the lack of proper initialization of _currentDefaultAdmin during contract deployment and highlights a gap in the role transfer logic that lead to unintended multi-admin privileges.

■ REMEDIATION

● FIXED

L-01

Pause bypass for approvals via permit

SEVERITY

● LOW

STATUS

● FIXED

■ DESCRIPTION

USHToken gates approve() with whenNotPaused, but inherits permit() from Solmate without overriding. While paused, anyone can still set/refresh allowances via permit, pre-arming drains that execute the instant the token is unpaused.

■ REMEDIATION

● FIXED

L-02

Burner role blocked by Auth Manager checks

SEVERITY

● LOW

STATUS

● FIXED

■ DESCRIPTION

USHToken.burn(from, amount) calls `_checkAuthTransfer(from, address(0))`. If `from` has since become banned/sanctioned (via `AuthManager`), `checkSanctioned/checkBanned` will revert. This prevents the protocol (holder of `BURNER_ROLE`) from programmatically burning balances from blocked accounts a common compliance/control action.

■ REMEDIATION

● FIXED

I-01

chainalysisOracleEnabled() function logic inverted

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

The view function `chainalysisOracleEnabled()` returns true when the oracle is unset (i.e., disabled), and false when the oracle is actually configured. This behavior contradicts the function name, which suggests it should return true when the oracle is enabled.

■ RECOMMENDATION

Fix logic:

```
function chainalysisOracleEnabled() public view returns (bool) {  
    return KycManagerStorageLib.getV1().chainalysisOracle != address(0);  
}
```

■ REMEDIATION

● FIXED

08 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits